FASE II DOCUMENTACIÓN PROYECTO S8

GUSTAVO ADOLFO CARDONA BAYONA

JONATHAN SOLER GUEVARA

HUGO ANDRÉS FORERO ANGEL

INGENIERÍA DE SOFTWARE, UNIVERSIDAD MANUELA BELTRÁN

SISTEMAS TRANSACCIONALES A1

JAMILTON FERNANDO BENVIDES

BOGOTÁ D.C.

18 de septiembre de 2026

TABLA DE CONTENIDO

1. INFORMACIÓN GENERAL DEL PROYECTO.....2
2. OBJETIVO CENTRAL.....5
3. OBJETIVOS ESPECÍFICOS.....6
4. ARQUITECTURA DE SOFTWARE.....7
5. DIAGRAMA DE CASOS DE USO UML.....11
6. DIAGRAMA DE CLASES UML.....16
7. MODELO RELACIONAL.....19
CONCLUSIONES.....24
REFERENCIAS.....25

1. INFORMACIÓN GENERAL DEL PROYECTO

En esta sección se retoma la contextualización elaborada en la Fase I y se presenta de manera deliberada sin decisiones técnicas: se describe qué problema existe, qué necesidad genera y qué propiedades debe cumplir la solución, en términos del negocio hotelero. Las decisiones sobre cómo se construirá el sistema, es decir, el estilo arquitectónico, las tecnologías, el motor de base de datos y los mecanismos de control de concurrencia, no hacen parte de la contextualización; se adoptan y se justifican a partir de la sección 4, una vez definidos los objetivos y las funcionalidades. Esta separación atiende la observación recibida sobre la Fase I y permite rastrear cada decisión de diseño hasta un elemento concreto del problema.

1.1 NOMBRE DEL SISTEMA

Hotel Reservas: aplicación móvil transaccional (APK) destinada a la administración de reservas de habitaciones en un hotel.

1.2 DESCRIPCIÓN GENERAL

Hotel Reservas consiste en un sistema de procesamiento de transacciones en línea concebido para utilizarse desde dispositivos móviles. La herramienta permitirá al personal de hoteles pequeños y medianos registrar huéspedes y habitaciones, verificar en tiempo real cuáles se encuentran disponibles, crear, actualizar o cancelar reservas, ingresar pagos, emitir comprobantes, gestionar el ingreso y la salida de los clientes y consultar informes de ocupación. La operación fundamental es la confirmación de una reserva, que debe comportarse como una

transacción con propiedades ACID sobre un recurso compartido: la disponibilidad de la habitación. De este modo, dos solicitudes simultáneas para el mismo alojamiento y periodo se resolverán de manera determinista, evitando que se produzcan reservas duplicadas (Elmasri y Navathe, 2016). Esta es una propiedad exigida a la solución y no una decisión de implementación; la manera de garantizarla se define en las secciones 4 y 7.

1.3 PROBLEMÁTICA O NECESIDAD IDENTIFICADA

Durante la Fase I se determinó que muchos hoteles pequeños y medianos administran sus reservas por medios manuales o parcialmente automatizados, entre ellos cuadernos de recepción, archivos compartidos, llamadas telefónicas y aplicaciones de mensajería. Como consecuencia, la información relacionada con cada habitación permanece distribuida entre distintos colaboradores y turnos, sin que exista un repositorio central confiable. Esta dispersión genera cuatro problemas principales: sobreventas o reservas dobles, pérdida de solicitudes, diferencias entre la reserva, el pago y la facturación, y falta de evidencia para atender reclamaciones. Por tanto, se requiere un registro centralizado, seguro y concurrente de la disponibilidad, donde las acciones de confirmar, modificar y cancelar se realicen de forma atómica, aislada y persistente, puedan ejecutarse desde cualquier área del hotel y dejen constancia de cada modificación.

1.4 JUSTIFICACIÓN

La propuesta resulta pertinente porque los inconvenientes identificados producen efectos económicos y reputacionales inmediatos. Una sobreventa puede obligar al hotel a trasladar al cliente o renunciar al ingreso; una reserva perdida puede dejar desocupada una habitación que habría podido venderse; y una inconsistencia en los registros demanda tiempo adicional para conciliar información. La implementación de un sistema transaccional previene reservas

duplicadas, concentra la información en una fuente única accesible desde los dispositivos móviles del personal, conserva el historial de cambios y disminuye las labores manuales. En el ámbito académico, la gestión hotelera constituye un ejemplo claro de procesamiento transaccional, ya que la secuencia de las operaciones y el manejo de la concurrencia repercuten directamente en los resultados económicos. Por ello, el proyecto permite llevar a la práctica los contenidos estudiados en la asignatura.

1.5 ALCANCE DE LA SOLUCIÓN

El diseño contempla un solo hotel, un inventario central de habitaciones, dos perfiles de usuario, recepcionista y administrador, y una habitación vinculada a cada reserva. Cubre el ciclo completo de la reserva, desde la consulta de disponibilidad hasta la salida del huésped, así como los informes básicos de ocupación. Se excluyen la conexión con pasarelas de pago, administradores de canales comerciales y plataformas contables externas; estas integraciones se consideran posibles ampliaciones futuras.

1.6 DELIMITACIÓN DE LAS FASES

La Fase I se limitó a la contextualización: describió la problemática de la gestión fragmentada de la disponibilidad, la necesidad de un registro único y concurrente, la justificación del proyecto y la pregunta problema. Su producto es la comprensión del problema y de las propiedades que debe tener cualquier solución, sin comprometer todavía una tecnología ni un estilo arquitectónico.

La presente Fase II corresponde al análisis y el diseño. Abarca la formulación de objetivos, el modelo de casos de uso, el diagrama de clases, el modelo relacional, la arquitectura de software y la relación trazable entre todos ellos. Es en esta fase donde se toman las decisiones

de diseño y se justifican una a una frente a la problemática: el estilo arquitectónico en la sección 4, la plataforma móvil, el lenguaje y el motor de base de datos en la sección 4.5, y los mecanismos de control de concurrencia en la sección 7.2. En esta fase no se desarrollarán la base de datos, la API ni la aplicación operativa, pues su implementación se reserva para las etapas de construcción y pruebas.

2. OBJETIVO CENTRAL

Diseñar una aplicación móvil transaccional que asegure la integridad, coherencia y trazabilidad de las reservas de habitaciones en hoteles pequeños o medianos. Las operaciones simultáneas de confirmación, modificación y cancelación deberán procesarse con propiedades ACID sobre una fuente central de disponibilidad, con el propósito de evitar las sobreventas y la pérdida de reservas detectadas en el diagnóstico.

3. OBJETIVOS ESPECÍFICOS

En primer lugar, definir y justificar una arquitectura de software que diferencie la interfaz de usuario, las reglas del negocio, la administración de transacciones, el acceso a la información y la base de datos. Así, el procesamiento transaccional permanecerá en un punto central y sus garantías no dependerán del equipo desde el cual se utilice la aplicación.

En segundo lugar, representar mediante casos de uso las funciones disponibles para el recepcionista y el administrador. El modelo debe comprender el registro de huéspedes y habitaciones, la consulta de disponibilidad, todas las etapas de una reserva, el control de entradas y salidas y la generación de informes de ocupación.

En tercer lugar, establecer la estructura estática de la solución a través de un diagrama de clases que incluya las entidades del negocio, sus características, operaciones y vínculos. Se dará especial relevancia al proceso de confirmación de reservas y al registro de auditoría.

En cuarto lugar, formular un modelo relacional con las tablas, claves, restricciones y cardinalidades que eviten reservas confirmadas superpuestas para una misma habitación y permitan gestionar apropiadamente la concurrencia, sobre el motor de base de datos que se seleccione y justifique en la arquitectura.

En quinto lugar, evidenciar la correspondencia entre el problema, los objetivos, las funciones, los casos de uso, las clases, el esquema relacional y la arquitectura, garantizando que todos los componentes del diseño se fundamenten en los elementos previamente definidos.

4. ARQUITECTURA DE SOFTWARE

A partir de esta sección el documento pasa de la contextualización a las decisiones de diseño. Cada decisión se presenta con su justificación y se vincula con el aspecto de la problemática o con el atributo de calidad que la motiva, de modo que sea posible distinguir lo que el problema exige de lo que el equipo eligió para resolverlo. Las decisiones que se toman aquí son cuatro: el estilo arquitectónico, la plataforma de la interfaz de usuario, la tecnología de la lógica de negocio y el motor de base de datos.

4.1 TIPO DE ARQUITECTURA SELECCIONADO

Se propone una arquitectura cliente-servidor estructurada en cuatro niveles: presentación, lógica de negocio, acceso a datos y almacenamiento. La presentación se ejecutará en el dispositivo móvil, mientras que los demás niveles funcionarán en el servidor. Cada capa se relacionará únicamente con la que se encuentra inmediatamente debajo mediante interfaces establecidas. Esta organización coincide con el patrón descrito por Bass et al. (2021), que limita las dependencias entre módulos a relaciones autorizadas entre capas consecutivas. En la presentación se empleará además el patrón MVVM Modelo, Vista y Modelo de Vista, recomendado por Microsoft (2024b) para el desarrollo con .NET MAUI.

4.2 DIAGRAMA DE LA ARQUITECTURA

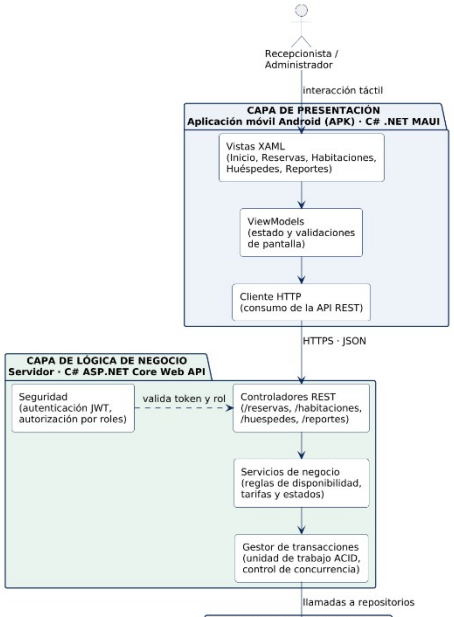


Figura 1 Arquitectura en capas del sistema Hotel Reservas

Nota. Creación propia. Las flechas muestran el sentido de las solicitudes y cada nivel depende exclusivamente del que se encuentra debajo.

4.3 COMPONENTES PRINCIPALES Y FUNCIÓN DE CADA UNO

La capa de presentación corresponde a la aplicación Android programada en C# mediante .NET MAUI y generada como archivo APK. Las vistas XAML representan las pantallas de inicio, reservas, habitaciones, huéspedes e informes disponibles en el repositorio del proyecto. Los modelos de vista administran el estado de las pantallas y validan el formato de la información antes de enviarla, mientras que el cliente HTTP centraliza el acceso a la API REST. Este nivel no implementa reglas del negocio ni se conecta directamente con la base de datos.

La lógica de negocio se implementará mediante una API REST creada en C# con ASP.NET Core Web API. Sus controladores publicarán los recursos de reservas, habitaciones, huéspedes, pagos e informes y transformarán las solicitudes HTTP en invocaciones a los servicios. Estos servicios contendrán las reglas del dominio, como el cálculo de la disponibilidad, la selección de tarifas y las transiciones válidas entre estados. A su vez, el gestor transaccional reunirá en una sola operación acciones como validar nuevamente la disponibilidad, crear la

reserva, registrar el pago y expedir el comprobante, aplicando el control de concurrencia dentro de una única transacción de base de datos. La seguridad validará la identidad mediante tokens JWT y limitará las acciones según el perfil asignado.

Para la capa de acceso a datos se utilizará Entity Framework Core como herramienta de mapeo objeto-relacional. Los repositorios proporcionarán las operaciones de consulta y almacenamiento de cada entidad, y DbContext controlará la correspondencia entre objetos y tablas, la conexión y el alcance de las transacciones. En situaciones que exijan un manejo más específico, como la utilización de sugerencias de bloqueo, se empleará ADO.NET junto con instrucciones T-SQL expresas.

La capa de almacenamiento estará conformada por una base de datos Microsoft SQL Server gestionada desde SQL Server Management Studio. Allí se ubicarán las tablas, claves, reglas de integridad, índices y registros de auditoría. Las transacciones ofrecidas por el motor garantizarán en última instancia el cumplimiento de las propiedades ACID (Microsoft, 2024a).

4.4 COMUNICACIÓN ENTRE COMPONENTES

La aplicación se conectará con la API por medio de solicitudes HTTPS cuyos contenidos se enviarán en JSON, conforme al enfoque REST. Cada recurso dispondrá de una ruta y se operará mediante los métodos GET, POST, PUT y DELETE. Las respuestas incluirán estados HTTP interpretados por la aplicación; por ejemplo, si se intenta asignar una habitación que acaba de ser reservada, se devolverá el código 409 conflicto acompañado de una explicación. En el servidor, los controladores llamarán a los servicios y estos al gestor transaccional mediante métodos directos e inyección de dependencias. El nivel de datos se comunicará con SQL Server a través del proveedor de .NET, que remitirá instrucciones T-SQL usando el protocolo TDS. La

unidad de trabajo comenzará con BEGIN TRANSACTION y terminará con COMMIT o ROLLBACK, según el resultado obtenido.

4.5 JUSTIFICACIÓN DE LA ARQUITECTURA Y DE LAS TECNOLOGÍAS

La organización por capas responde a la necesidad detectada porque sitúa el procesamiento transaccional en el servidor y en la base de datos. En consecuencia, todos los teléfonos consultan y actualizan el mismo registro de disponibilidad, superando la dispersión que ocasiona las reservas dobles. Esta separación también permite modificar o ampliar la interfaz móvil sin cambiar las reglas del negocio, habilitar en el futuro el consumo de la API desde un portal web o un gestor de canales y sustituir o actualizar el motor de datos sin alterar la aplicación. En cuanto a calidad, facilita la modificación y las pruebas aisladas de los componentes, y fortalece la seguridad al centralizar la autenticación y la autorización en un único acceso (Bass et al., 2021). A diferencia de una solución con almacenamiento local, conserva la interacción concurrente entre usuarios, requisito esencial del proyecto.

Las decisiones tecnológicas se justifican de la misma manera. La interfaz se construye como aplicación móvil Android, distribuida en formato APK, porque la necesidad establece que el personal debe consultar y confirmar disponibilidad desde el lugar del hotel donde se encuentre, y porque el docente de la asignatura recomendó esta modalidad; se descartó una base de datos embebida en el dispositivo (SQLite) porque rompería el registro único y, con él, la garantía frente a la doble reserva. Se elige C# con .NET MAUI para la aplicación y C# con ASP.NET Core Web API para la lógica de negocio porque permiten usar un solo lenguaje en las capas de presentación y de servidor, cuentan con soporte del fabricante y se integran de forma nativa con el motor de datos elegido. Se elige Microsoft SQL Server porque ofrece transacciones explícitas, niveles de aislamiento configurables, bloqueos de fila y el tipo ROWVERSION, que

son precisamente los mecanismos que la sección 7.2 emplea para impedir el solapamiento de reservas (Microsoft, 2024a); se descartó MySQL por decisión del equipo y se consideró PostgreSQL como alternativa equivalente, sin ventaja decisiva para este alcance. Ninguna de estas decisiones altera la contextualización de la Fase I; todas responden a ella.

5. DIAGRAMA DE CASOS DE USO UML

5.1 ACTORES

Pressman y Maxim (2021) señalan que un caso de uso explica la manera en que un actor interactúa con el sistema para lograr un propósito; por ello, el modelado comienza con la identificación de quienes participan. La propuesta contempla dos actores humanos. El recepcionista desempeña las tareas operativas: atender clientes, verificar disponibilidad, crear, modificar o cancelar reservas, registrar pagos y controlar el ingreso y la salida. El administrador mantiene el inventario de habitaciones y tarifas, gestiona las cuentas de usuario y consulta los informes de ocupación. Los dos perfiles deben autenticarse antes de acceder a las funciones. No se incorporan actores externos porque esta fase no considera servicios de pago ni plataformas de terceros.

5.2 DIAGRAMA

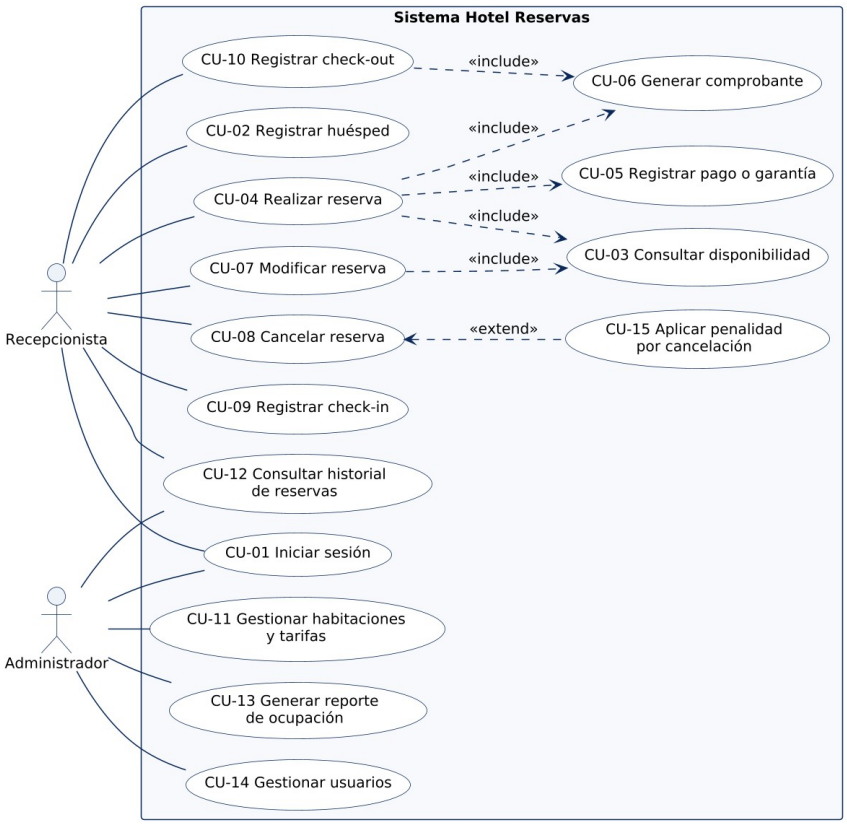


Figura 2 Diagrama de casos de uso del sistema Hotel Reservas

Nota. Diseño propio basado en UML 2.5 (Object Management Group, 2017). El contorno rectangular delimita el sistema; «include» representa una función obligatoria que se reutiliza y «extend» identifica un comportamiento opcional condicionado.

CU-04 y CU-07 incorporan CU-03, Consultar disponibilidad porque una reserva no puede registrarse ni modificarse sin comprobar previamente que la habitación esté libre durante el periodo seleccionado. Esta consulta también puede ser ejecutada directamente por el recepcionista. CU-04 incluye CU-05 y CU-06, puesto que la confirmación requiere registrar el pago o la garantía y producir el comprobante como parte de una misma transacción. Por otra parte, CU-15 amplía CU-08 únicamente cuando la cancelación se presenta después del plazo admitido por las políticas del establecimiento.

5.3 DESCRIPCIÓN DE LOS PRINCIPALES CASOS DE USO

Nombre: CU-04 Crear una reserva.

Actor principal: Recepcionista.

Objetivo: Formalizar la reserva de una habitación para un huésped durante unas fechas determinadas, sin permitir que el mismo espacio sea asignado a otra solicitud.

Precondiciones: El recepcionista se encuentra autenticado. El huésped ya está registrado o se incorpora durante el proceso mediante CU-02. Debe existir, como mínimo, una habitación perteneciente a la categoría requerida.

Flujo principal: El recepcionista especifica la categoría de habitación, las fechas de llegada y salida y la cantidad de personas. La plataforma ejecuta CU-03, presenta las alternativas

libres y muestra sus tarifas. Después de elegir una habitación y vincular al huésped, el sistema abre la transacción, vuelve a comprobar la disponibilidad con bloqueo, genera la reserva con estado confirmada, registra el pago o la garantía por medio de CU-05, produce el comprobante en CU-06, actualiza la bitácora y confirma la operación. Finalmente, informa el código asignado y presenta el comprobante.

Flujo alternativo: Cuando la segunda validación detecta que otra transacción ya ocupó la habitación, todas las acciones se revierten y se informa “habitación no disponible”, sin almacenar datos incompletos.

Resultado esperado: La reserva queda confirmada, vinculada con el pago y el comprobante, almacenada como una unidad indivisible y disponible inmediatamente para todos los usuarios.

Nombre: CU-03 Consultar disponibilidad.

Actor principal: Recepcionista.

Objetivo: Identificar las habitaciones disponibles de una categoría determinada dentro de un intervalo de fechas.

Precondiciones: El usuario inició sesión, las fechas ingresadas son correctas y la salida se programó después de la entrada.

Flujo principal: El usuario selecciona el tipo de habitación y el periodo. El sistema localiza aquellas unidades que no estén en mantenimiento ni tengan reservas confirmadas o huéspedes alojados en fechas que se superpongan con la consulta. Luego presenta los resultados, la tarifa nocturna aplicable y el costo total aproximado.

Resultado esperado: Se obtiene una relación de habitaciones libres calculada con datos del repositorio central, no con registros almacenados localmente.

Nombre: CU-08 Cancelar reserva.

Actor principal: Recepcionista.

Objetivo: Dejar sin efecto una reserva confirmada y devolver la habitación al inventario disponible.

Precondiciones: La reserva está registrada y conserva el estado confirmado. El recepcionista se encuentra autenticado.

Flujo principal: El recepcionista localiza la reserva mediante su código o los datos del cliente y registra la causa de la cancelación. El sistema revisa la política aplicable y, si corresponde, activa CU-15 para calcular la penalidad. Posteriormente abre una transacción, cambia la reserva a cancelada, registra la devolución completa o parcial del pago, actualiza la bitácora y confirma los cambios. Las fechas asociadas a la habitación quedan nuevamente disponibles.

Resultado esperado: La reserva figura como cancelada, el pago se revierte conforme a la política y la habitación se libera. El historial conserva la identidad del responsable y el momento de la operación.

Nombre: CU-09 Registrar check-in.

Actor principal: Recepcionista.

Objetivo: Confirmar la llegada del cliente y registrar que la habitación se encuentra efectivamente ocupada.

Precondiciones: Hay una reserva confirmada cuya entrada corresponde al día en curso y la habitación aparece con estado reservada.

Flujo principal: El recepcionista consulta la reserva, valida la identidad del huésped y confirma su ingreso. Dentro de una sola transacción, el sistema cambia la reserva al estado en casa, marca la habitación como ocupada y guarda el movimiento en la bitácora.

Resultado esperado: La reserva queda en estado en casa, la habitación se registra como ocupada y se almacena la hora de llegada.

Nombre: CU-13 Generar reporte de ocupación.

Actor principal: Administrador.

Objetivo: Consultar la ocupación efectiva y estimada del establecimiento durante un intervalo determinado.

Precondiciones: El administrador está autenticado y existen reservas en el sistema.

Flujo principal: El administrador define el periodo de análisis. Para cada fecha, el sistema determina cuántas habitaciones están ocupadas, reservadas o disponibles y calcula el porcentaje de ocupación utilizando las reservas confirmadas, en casa y cerradas. Los resultados se visualizan en pantalla.

Resultado esperado: Se genera un informe de ocupación acorde con la situación real de las reservas, sin necesidad de realizar conciliaciones manuales.

6. DIAGRAMA DE CLASES UML

El modelo de clases muestra la organización estática necesaria para ejecutar los casos de uso definidos. Sus entidades representan los conceptos identificados en esos procesos: huésped, habitación, categoría de habitación, tarifa, reserva, pago, comprobante y bitácora. Usuario se plantea como clase abstracta y se especializa en Recepcionista y Administrador; además de representar los perfiles del sistema, permite reconocer al responsable de cada acción. ServicioReservas corresponde a la lógica de negocio y no a una entidad del dominio. Se incorpora porque coordina la operación crítica de confirmación.

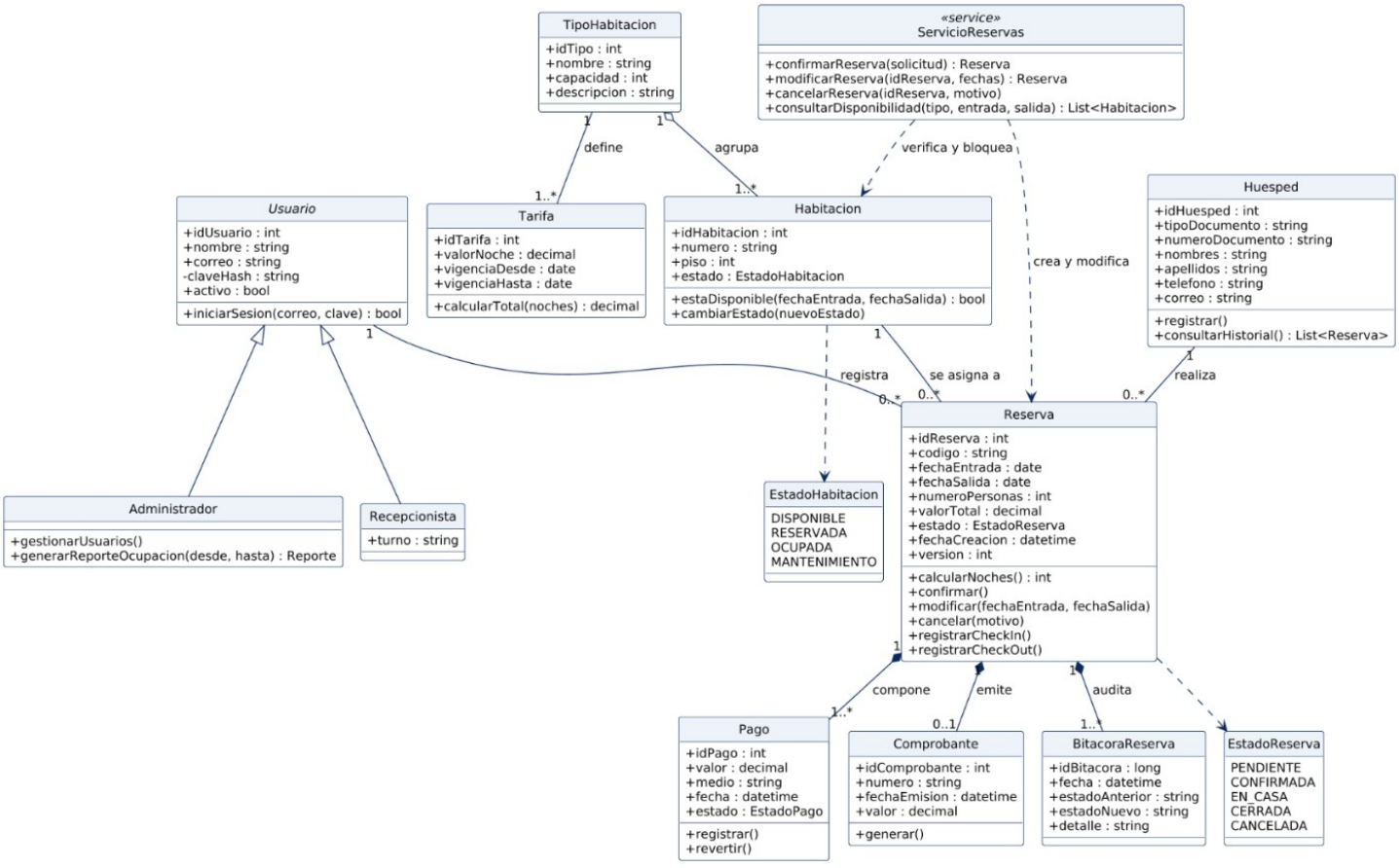


Figura 3 Diagrama de clases del sistema Hotel Reservas

Nota. Elaboración propia con UML 2.5 (Object Management Group, 2017). El diamante sólido expresa composición, el diamante sin relleno representa agregación, las flechas con extremo vacío señalan herencia y los trazos discontinuos indican dependencia.

6.1 RELACIONES Y MULTIPLICIDADES

Cada tipo reúne una o varias habitaciones y dispone de una o más tarifas con periodos definidos. La asociación se representa como agregación, pues las habitaciones pueden existir aunque se reorganice su clasificación. Un huésped puede no tener reservas o registrar varias, mientras que toda reserva debe corresponder a un único huésped. Una habitación admite múltiples reservas en distintos momentos, pero cada reserva se asigna exactamente a una habitación. Asimismo, cada usuario puede registrar ninguna o muchas reservas, lo que permite establecer quién creó cada registro.

La composición indica que el pago, el comprobante y la bitácora dependen de la reserva y carecen de sentido sin ella, por lo que desaparecen cuando esta se elimina. Toda reserva contará al menos con un pago la garantía inicial y podrá acumular otros movimientos, como abonos, saldos o devoluciones. También podrá tener un solo comprobante como máximo y deberá incluir al menos una anotación de bitácora correspondiente a su creación. La generalización de Usuario en Recepcionista y Administrador permite reutilizar el mecanismo de autenticación y, al mismo tiempo, distinguir los permisos de cada perfil.

6.2 CORRESPONDENCIA CON LOS CASOS DE USO

CU-04 se materializa mediante confirmarReserva, método de ServicioReservas que consulta estaDisponible en Habitacion, crea la Reserva, registra el Pago, produce el Comprobante y añade la entrada en BitacoraReserva. CU-03 se relaciona con

consultarDisponibilidad, operación que revisa las habitaciones de una categoría y aplica estaDisponible. CU-08 se ejecuta con cancelarReserva, que utiliza cancelar en Reserva y revertir en Pago. CU-09 y CU-10 corresponden respectivamente a registrarCheckIn y registrarCheckOut de Reserva, y ambos actualizan Habitacion mediante cambiarEstado. CU-13 se vincula con generarReporteOcupacion de Administrador. Finalmente, el campo version de Reserva contribuye al control de concurrencia optimista detallado en el esquema relacional.

7. MODELO RELACIONAL

El esquema relacional convierte las clases persistentes en tablas de Microsoft SQL Server, transforma sus atributos en columnas con tipos de datos definidos y ubica las claves foráneas en el extremo de multiplicidad “muchos” de cada relación, de acuerdo con las reglas propuestas por Elmasri y Navathe (2016). La jerarquía de Usuario se almacena en una sola tabla denominada USUARIO, acompañada por una columna rol, debido a que sus especializaciones incorporan pocos atributos adicionales. Las enumeraciones se guardan como cadenas breves sujetas a validaciones CHECK. ServicioReservas no se convierte en tabla porque pertenece a la lógica de negocio.

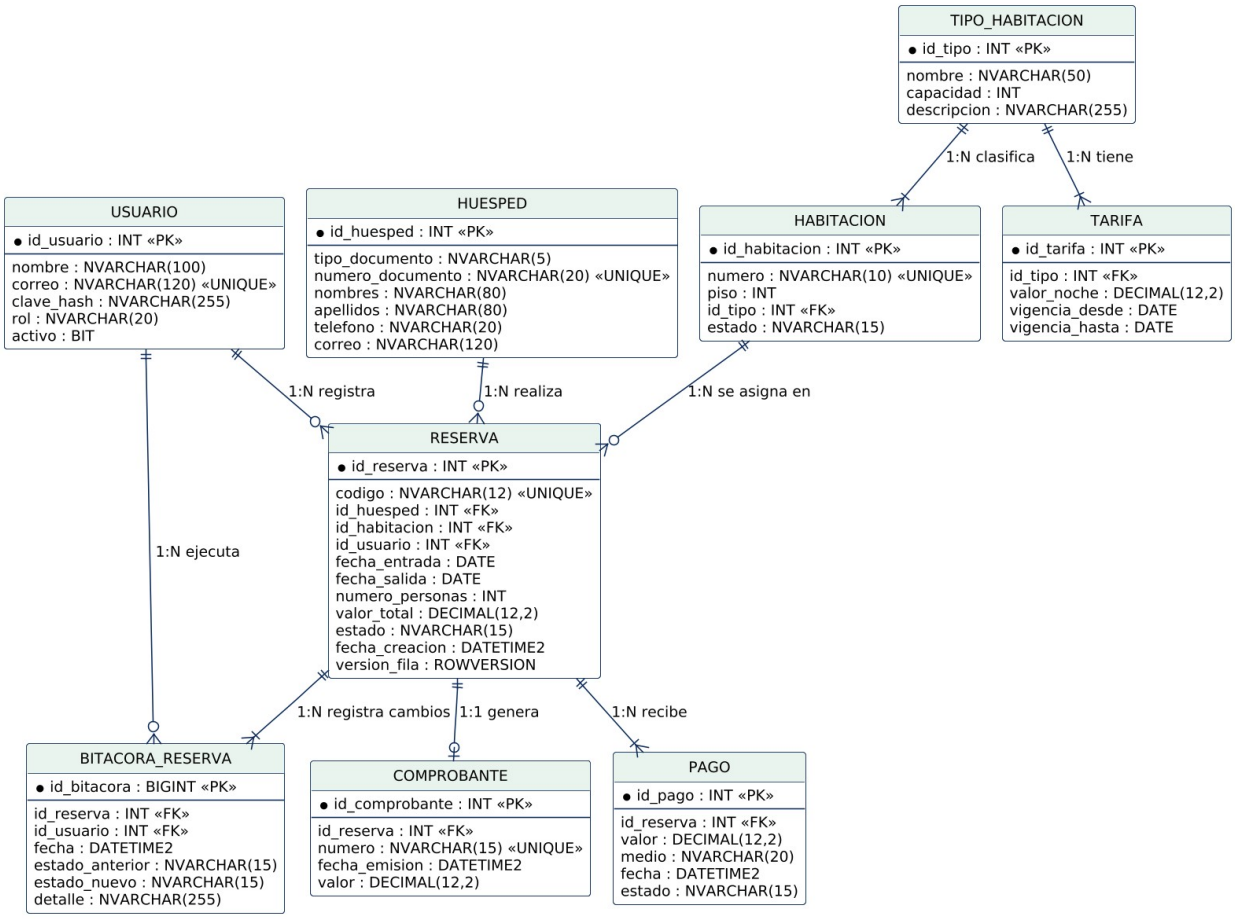


Figura 4 Modelo relacional del sistema Hotel Reservas

Nota. Construcción propia mediante notación de pata de gallo. PK corresponde a clave primaria, FK a clave foránea y UNIQUE a una condición de valor único. Cada etiqueta señala la cardinalidad de la relación.

7.1 DESCRIPCIÓN DE LAS TABLAS

USUARIO conserva las credenciales del personal, su perfil RECEPCIONISTA o ADMINISTRADOR, el correo irrepetible y la contraseña protegida mediante hash. HUESPED reúne los datos de identificación y contacto de los clientes y evita documentos duplicados. TIPO_HABITACION organiza las unidades por denominación y capacidad; TARIFA guarda el precio nocturno de cada categoría y su periodo de aplicación, de manera que se mantenga el historial de valores. HABITACION representa el inventario físico e incluye número único, piso, tipo y situación actual.

RESERVA constituye la tabla principal. Contiene referencias al huésped, la habitación y el usuario responsable, así como las fechas de llegada y salida, la cantidad de ocupantes, el importe calculado con la tarifa vigente, el estado y el momento de creación. SQL Server modifica automáticamente version_fila, columna ROWVERSION utilizada para la concurrencia optimista: el sistema remite la versión previamente consultada y solo permite la actualización cuando todavía coincide con la almacenada. PAGO almacena cada transacción monetaria de la reserva, junto con su forma de pago y condición registrado o revertido. COMPROBANTE conserva el soporte generado al confirmar, identificado con numeración única. BITACORA_RESERVA documenta cada cambio de estado, el usuario que lo efectuó, la fecha y los estados anterior y posterior, aportando la evidencia necesaria frente a posibles reclamaciones.

7.2 RESTRICCIONES QUE GARANTIZAN LA INTEGRIDAD TRANSACCIONAL

Las claves foráneas evitan crear reservas sin huésped, habitación o usuario, y también impiden pagos, comprobantes o movimientos de bitácora desvinculados de una reserva. Mediante restricciones CHECK se valida que la salida sea posterior a la entrada, que la cantidad de personas no exceda la capacidad establecida y que el estado pertenezca al conjunto autorizado. La regla principal —evitar dos reservas activas superpuestas para la misma habitación— se respalda con un índice sobre (id_habitacion, fecha_entrada, fecha_salida) y una comprobación ejecutada dentro de la transacción de confirmación. Para ello se utiliza el aislamiento SERIALIZABLE o las sugerencias UPDLOCK y HOLDLOCK sobre las filas correspondientes, haciendo que dos operaciones simultáneas sobre la misma habitación se atiendan secuencialmente y solo una sea confirmada (Microsoft, 2024a). El fragmento siguiente ejemplifica la transacción prevista para la etapa de construcción.

```
      BEGIN TRANSACTION;
  IF EXISTS (SELECT 1 FROM RESERVA WITH (UPDLOCK, HOLDLOCK)
             WHERE id_habitacion = @hab
               AND estado IN ('CONFIRMADA','EN_CASA')
               AND fecha_entrada < @salida AND fecha_salida > @entrada)
    BEGIN ROLLBACK TRANSACTION; THROW 50001, 'Habitación no disponible', 1; END
  INSERT INTO RESERVA (...) VALUES (...);
  INSERT INTO PAGO (...) VALUES (...);
  INSERT INTO COMPROBANTE (...) VALUES (...);
  INSERT INTO BITACORA_RESERVA (...) VALUES (...);
COMMIT TRANSACTION;
```

8. INTEGRACIÓN DE LOS MODELOS

La documentación se desarrolló siguiendo una secuencia coherente. La dispersión de los datos y las reservas duplicadas identificadas en el problema condujeron al objetivo general de asegurar integridad, consistencia y trazabilidad en condiciones de concurrencia. Posteriormente, este propósito se dividió en objetivos específicos asociados con la arquitectura, los casos de uso, las clases, el esquema relacional y la trazabilidad. Cada función fue representada por un caso de uso; cada caso se sustenta en determinadas clases; cada clase persistente se refleja en una tabla; y la arquitectura establece el componente donde debe procesarse cada acción. La Tabla 1 sintetiza estas relaciones para las funciones más importantes.

Tabla 1 Matriz de trazabilidad entre problemática, objetivos, casos de uso, clases, tablas y componentes

Problema	Objetivo	Caso de uso	Clases	Tablas	Componente
Reservas duplicadas por validación sin control	1, 3, 4	CU-03, CU-04	ServicioReservas, Habitacion, Reserva, Tarifa	RESERVA, HABITACION, TARIFA	Gestión transaccional y base de datos mediante aislamiento y bloqueos
Diferencias entre reserva, pago y soporte	3, 4	CU-04, CU-05, CU-06	Reserva, Pago, Comprobante	RESERVA, PAGO, COMPROBANTE	Gestor transaccional como unidad de

					trabajo indivisible
Pérdida o modificación de reservas sin seguimiento	2, 3	CU-07, CU-08, CU-15	Reserva, Pago, ServicioReservas	RESERVA, PAGO	Servicios del negocio y repositorios
Información desactualizada sobre las habitaciones	2, 3	CU-09, CU-10, CU-11	Habitacion, Reserva, TipoHabitacion	HABITACION, RESERVA, TIPO_HABITACION	Servicios del negocio y DbContext
Ausencia de evidencia para atender reclamaciones	3, 4	CU-04, CU-07, CU-08, CU-09, CU-10	BitacoraReserva, Usuario	BITACORA_RESERVA, USUARIO	Gestor transaccional y seguridad para identificar al usuario
Datos dispersos para apoyar decisiones	2	CU-12, CU-13	Administrador, Reserva, Huesped	RESERVA, HABITACION, HUESPED	Controladores y servicios destinados a reportes
Acceso a los datos sin autorización suficiente	1, 2	CU-01, CU-14	Usuario, Recepcionista, Administrador	USUARIO	Seguridad basada en JWT y perfiles

Nota. Elaboración propia. La numeración de los objetivos específicos sigue el orden presentado en el apartado 3.

La matriz confirma que los componentes fueron concebidos de forma integrada. La confirmación transaccional, respuesta principal ante las reservas dobles, se identifica como CU-04; se coordina desde ServicioReservas usando Reserva, Pago y Comprobante; se guarda en las tablas equivalentes bajo las reglas expuestas en el apartado 7.2; y se procesa en el gestor transaccional de la lógica de negocio con el respaldo del control de concurrencia del motor de datos. De manera semejante, la necesidad de seguimiento se concreta en la clase y la tabla de bitácora, mientras que el módulo de seguridad permite reconocer al usuario responsable de cada movimiento.

CONCLUSIONES

El trabajo de análisis y diseño de esta etapa convirtió la propuesta formulada en la Fase I en una solución organizada. El objetivo general responde directamente al problema y los cinco objetivos específicos se desarrollan dentro del documento. La arquitectura concentra las transacciones en el servidor y la base de datos; el modelo de casos de uso reúne quince funciones distribuidas entre dos actores y sitúa la confirmación de reservas como proceso central; el diagrama de clases estructura esas funciones por medio de las entidades, relaciones y operaciones del dominio; el modelo relacional las traduce en nueve tablas de SQL Server con claves y reglas destinadas a impedir superposiciones; y la matriz demuestra la correspondencia entre las decisiones de diseño y las necesidades detectadas. En síntesis, la prevención de reservas duplicadas queda incorporada a la solución técnica y deja de depender únicamente del cuidado de los colaboradores.

También se establecen las actividades que deberán abordarse en las etapas siguientes. Primero será necesario implementar la base de datos, preparar los scripts de tablas, restricciones e índices y comprobar la confirmación transaccional mediante dos sesiones concurrentes. Después deberá construirse la API con ASP.NET Core, incluidos controladores, servicios y gestión de transacciones, y decidir con base en pruebas entre el bloqueo pesimista mediante UPDLOCK y HOLDLOCK y la concurrencia optimista basada en ROWVERSION. A continuación, se conectará la aplicación móvil existente con la API y se desarrollarán gradualmente los casos de uso, empezando por CU-01, CU-03 y CU-04. Por último, deberán definirse las pruebas de aceptación y los indicadores de calidad que permitan verificar que la implementación satisface lo especificado en este diseño.

REFERENCIAS

Bass, L., Clements, P. y Kazman, R. (2021). *Software architecture in practice* (4.ª ed.). Addison-Wesley.

Elmasri, R. y Navathe, S. B. (2016). *Fundamentals of database systems* (7.ª ed.). Pearson.

Microsoft. (2024a). *Transacciones (Transact-SQL)*. Microsoft Learn. https://learn.microsoft.com/es-es/sql/t-sql/language-elements/transactions-transact-sql

Microsoft. (2024b). *Model-View-ViewModel (MVVM)*. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/architecture/maui/mvvm

Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), version 2.5.1*. https://www.omg.org/spec/UML/2.5.1

Pressman, R. S. y Maxim, B. R. (2021). *Ingeniería del software: un enfoque práctico* (9.ª ed.). McGraw-Hill.